

Roll No. : 2520030027

Name : Venkata Sai Krishna

SEC : 8

PRACTICAL-2:

Aim: Develop a C program that uses the system calls open(), read(), write(), and close() to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Use the strace utility to trace all system calls generated by the command cat sample.txt. Analyze the sequence of system calls and identify the kernel services involved.

Procedure :

```
venkatsaikrishna5@VSK-HP:~$ cd ~
venkatsaikrishna5@VSK-HP:~$ mkdir practical02
mkdir: practical02: File exists
venkatsaikrishna5@VSK-HP:~$ cd practical02
venkatsaikrishna5@VSK-HP:~/practical02$ pwd
/home/venkatsaikrishna5/practical02
venkatsaikrishna5@VSK-HP:~/practical02$ echo "Hello World" > sample.txt
venkatsaikrishna5@VSK-HP:~/practical02$ echo "This is my operating systems assignment." >> sample.txt
venkatsaikrishna5@VSK-HP:~/practical02$ cat sample.txt
Hello World
This is my operating systems assignment.
This file will be copied using system calls.
venkatsaikrishna5@VSK-HP:~/practical02$ nano copy.c
venkatsaikrishna5@VSK-HP:~/practical02$ gcc copy.c -o copy
venkatsaikrishna5@VSK-HP:~/practical02$ ./copy
File copied successfully.
venkatsaikrishna5@VSK-HP:~/practical02$ cat copy.txt
Hello World
This is my operating systems assignment.
This file will be copied using system calls.
venkatsaikrishna5@VSK-HP:~/practical02$ diff sample.txt copy.txt
```

```

venkatsaikrishna5@VSK-HP:~/practical02$ strace cat sample.txt
execve("/usr/bin/cat", ["cat", "sample.txt"], 0x7ffdef634768 /*
 26 vars */) = 0
brk(NULL)                                = 0x5963a08aa000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOU
S, -1, 0) = 0x753e82e6d000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such fi
le or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=20087, ...}) = 0
mmap(NULL, 20087, PROT_READ, MAP_PRIVATE, 3, 0) = 0x753e82e6800
0
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libselinux.so.1", O
_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0
\0\0\0\0"... , 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=199120, ...}) = 0
mmap(NULL, 206288, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0)
= 0x753e82e35000
mmap(0x753e82e3c000, 135168, PROT_READ|PROT_EXEC, MAP_PRIVATE|M
AP_FIXED|MAP_DENYWRITE, 3, 0x7000) = 0x753e82e3c000
mmap(0x753e82e5d000, 28672, PROT_READ, MAP_PRIVATE|MAP_FIXED|MA
P_DENYWRITE, 3, 0x28000) = 0x753e82e5d000
mmap(0x753e82e64000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MA
P_FIXED|MAP_DENYWRITE, 3, 0x2f000) = 0x753e82e64000
mmap(0x753e82e66000, 5584, PROT_READ|PROT_WRITE, MAP_PRIVATE|MA
P_FIXED|MAP_ANONYMOUS, -1, 0) = 0x753e82e66000
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libgcc_s.so.1", O_R
DONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0
\0\0\0\0"... , 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=187120, ...}) = 0
mmap(NULL, 185256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0)
= 0x753e82e07000

```

```

mmap(0x753e82e0b000, 147456, PROT_READ|PROT_EXEC, MAP_PRIVATE|M
AP_FIXED|MAP_DENYWRITE, 3, 0x4000) = 0x753e82e0b000
mmap(0x753e82e2f000, 16384, PROT_READ, MAP_PRIVATE|MAP_FIXED|MA
P_DENYWRITE, 3, 0x28000) = 0x753e82e2f000
mmap(0x753e82e33000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MA
P_FIXED|MAP_DENYWRITE, 3, 0x2c000) = 0x753e82e33000
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libm.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0
\0\0\0\0"... , 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=1198376, ...}) = 0
mmap(NULL, 1200152, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0)
= 0x753e82ce1000
mmap(0x753e82cf3000, 655360, PROT_READ|PROT_EXEC, MAP_PRIVATE|M
AP_FIXED|MAP_DENYWRITE, 3, 0x12000) = 0x753e82cf3000
mmap(0x753e82d93000, 466944, PROT_READ, MAP_PRIVATE|MAP_FIXED|M
AP_DENYWRITE, 3, 0xb2000) = 0x753e82d93000
mmap(0x753e82e05000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MA
P_FIXED|MAP_DENYWRITE, 3, 0x123000) = 0x753e82e05000
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0 \250\2\
0\0\0\0\0"... , 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0
\0\0\0\0"... , 840, 64) = 840
fstat(3, {st_mode=S_IFREG|0755, st_size=2186512, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0
\0\0\0\0"... , 840, 64) = 840
mmap(NULL, 2231696, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0)
= 0x753e82a00000
mmap(0x753e82a28000, 1671168, PROT_READ|PROT_EXEC, MAP_PRIVATE|
MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x753e82a28000
mmap(0x753e82bc0000, 319488, PROT_READ, MAP_PRIVATE|MAP_FIXED|M
AP_DENYWRITE, 3, 0x1c0000) = 0x753e82bc0000

```

```

venkatsaikrishna5@VSK-HP:~/practical02$ strace -o trace.txt cat
sample.txt
Hello World
This is my operating systems assignment.
This file will be copied using system calls.
venkatsaikrishna5@VSK-HP:~/practical02$ grep -E "open|read|write|close|execve|exit" trace.txt
execve("/usr/bin/cat", ["cat", "sample.txt"], 0x7fff9f93df08 /*
  26 vars */) = 0
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0"..., 832) = 832
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libgcc_s.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0"..., 832) = 832
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libm.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0"..., 832) = 832
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0"..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"..., 840, 64) = 840
pread64(3, "\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"..., 840, 64) = 840
close(3) = 0

```

```

close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libpcre2-8.so.0", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0"..., 832) = 832
close(3) = 0
openat(AT_FDCWD, "/proc/filesystems", O_RDONLY|O_CLOEXEC) = 3
read(3, "nodev\tsysfs\nnodev\ttmpfs\nnodev\tbd"..., 1024) = 442
close(3) = 0
openat(AT_FDCWD, "/proc/mounts", O_RDONLY|O_CLOEXEC) = 3
read(3, "none /usr/lib/modules/6.18.33.2-"..., 1024) = 1024
read(3, "e 0 0\ndevpts /dev/pts devpts rw,"..., 1024) = 1024
read(3, "c,relatime 0 0\nhugetlbfs /dev/hu"..., 1024) = 1024
read(3, "s /mnt/wslg/run/user/1000 tmpfs "..., 1024) = 118
read(3, "", 1024) = 0
close(3) = 0
openat(AT_FDCWD, "/proc/self/maps", O_RDONLY|O_CLOEXEC) = 3
read(3, "5ad9ebbf000-5ad9ebcf1000 r--p 0"..., 1024) = 1024
read(3, "3bfe000-7838d3bff000 r--p 000a90"..., 1024) = 1024
read(3, "00 08:30 13650"..., 1024) = 1024
read(3, " /usr/lib/x86_64-lin"..., 1024) = 1024
read(3, "0 13644 /us"..., 1024) = 566
close(3) = 0
openat(AT_FDCWD, "/usr/share/coreutils/locales/uucore/en-US.ftl", O_RDONLY|O_CLOEXEC) = 3
read(3, "# Common strings shared across a"..., 4080) = 4080
read(3, "", 32) = 0
close(3) = 0
openat(AT_FDCWD, "/usr/share/coreutils/locales/cat/en-US.ftl", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "sample.txt", O_RDONLY|O_CLOEXEC) = 3
close(5) = 0
close(4) = 0
close(3) = 0
exit_group(0) = ?
+++ exited with 0 +++

```

```
venkatsaikrishna5@VSK-HP:~/practical02$ cat sample.txt
Hello World
This is my operating systems assignment.
This file will be copied using system calls.
```

System Call	What it does
<code>execve()</code>	Loads and starts the <code>cat</code> program
<code>openat()</code>	Opens <code>sample.txt</code>
<code>read()</code>	Reads data from the file
<code>write()</code>	Displays data on the terminal
<code>close()</code>	Closes the file descriptor
<code>exit_group()</code>	Terminates the process

Kernel services involved

1. Process Management

`execve()` and `exit_group()` are related to process execution and termination.

2. File Management

`openat()` and `close()` manage file descriptors and files.

3. I/O Management

`read()` and `write()` handle data transfer between the program and the kernel.

Code:

```
GNU nano 8.7.1          copy.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int source, destination;
    char buffer[1024];
    ssize_t bytesRead;

    // Open source file
    source = open("sample.txt", O_RDONLY);

    if (source == -1)
    {
        perror("Error opening sample.txt");
        return 1;
    }

    // Open/Create destination file
    destination = open("copy.txt", O_WRONLY | O_CREAT | O_TRUNC>

    if (destination == -1)
    {
        perror("Error opening copy.txt");
        close(source);
        return 1;
    }

    // Copy contents
    while ((bytesRead = read(source, buffer, sizeof(buffer))) >
    {
        write(destination, buffer, bytesRead);
    }

    // Close files
    close(source);
    close(destination);

    printf("File copied successfully.\n");

    return 0;
}
```

Observation :

- The source file was successfully opened using `open()`.
- File contents were read using `read()` and written to the destination file using `write()`.
- `close()` successfully closed both files after copying.
- During execution, control transitioned from **user space to kernel space** whenever a system call was invoked.
- `strace` successfully traced the system calls generated by `cat sample.txt`.
- System calls such as `openat()`, `read()`, `write()`, and `close()` were observed.
- The kernel provided services for file access, reading data, writing output, and closing file descriptors.

Result :

The file was successfully copied using the **`open()`, `read()`, `write()`, and `close()` system calls**. The `strace` utility successfully traced the system calls generated by `cat sample.txt`, demonstrating how user-space programs interact with **kernel services** through system calls.